



ARQUITECTURA Y SISTEMAS OPERATIVOS

CLASE 6

SINCRONIZACIÓN DE PROCESOS



La Concurrency como origen de la Sincronización

“Concurrency no implica automáticamente correcta cooperación”

La necesidad de sincronización **aparece como consecuencia** directa de la concurrency.

Si varios procesos pueden estar activos al mismo tiempo, el sistema alterna rápidamente entre ellos para dar la sensación de simultaneidad. Y en sistemas multinúcleo, esa simultaneidad **puede ser real**.

No siempre existe un único camino lineal de ejecución. Hay múltiples flujos de instrucciones que pueden entrelazarse.

Si esos flujos son independientes, no hay problemas. Pero si comparten recursos, entonces el orden en que acceden a esos recursos sí importa.

Ese orden puede cambiar según la planificación de la CPU, el instante en que ocurre una interrupción, la duración de una operación de E/S o la ejecución paralela en distintos núcleos.

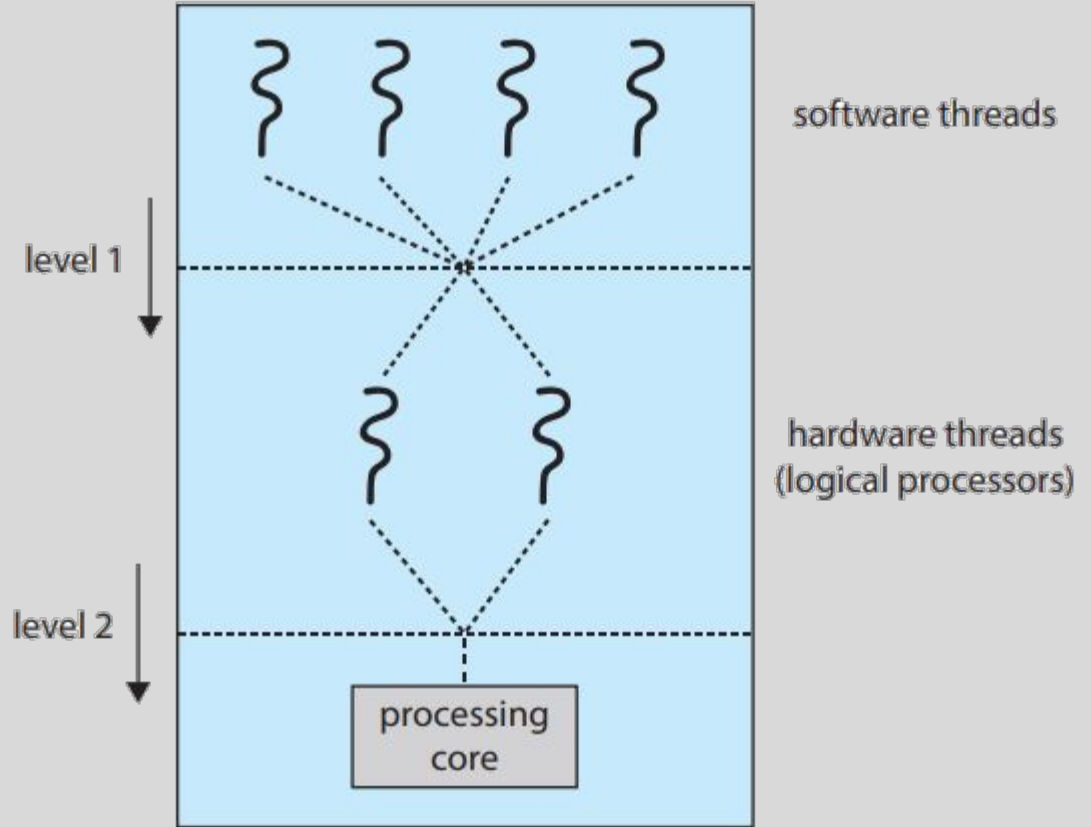
El Objetivo de la Sincronización

Objetivo: actuar como control de tráfico.

Su función es proteger las **regiones críticas** del código (donde se accede al recurso compartido), garantizando que ningún proceso interfiera con la lógica de otro mientras se manipulan datos críticos.

Planificación decide quién corre.

Sincronización decide cómo evitar que se molesten entre sí.



Two levels of scheduling



BIBLIOGRAFIA

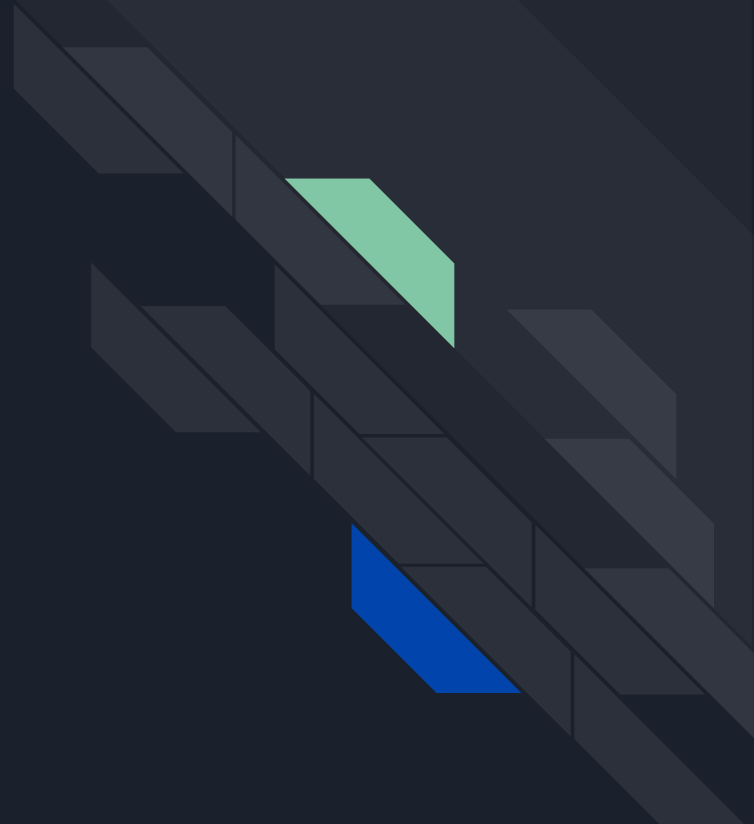
- Operating System Concepts. By Abraham, Silberschatz.
 - Capítulo VI
- Modern Operating Systems. By Andrew S. Tanenbaum.
 - Capítulo II

Temas de la Clase

- Background
- Race conditions
- Regiones críticas
- Exclusión mutua
- Busy waiting
- Peterson
- Soporte de hardware
- Sleep / Wakeup
- Semáforos
- Mutex
- Pthreads



BACKGROUND





Procesos cooperativos y datos compartidos

“Cuando hay cooperación, aparece la necesidad de coordinación.”

Un proceso es **cooperativo** cuando puede afectar a otros procesos o verse afectado por ellos.

Cuando existe información compartida, la ejecución de un proceso deja de ser completamente independiente. Lo que haga uno puede modificar el contexto en el que se ejecuta el otro.

Como el SO intercala la ejecución de procesos y no garantiza un orden fijo entre todas sus instrucciones, ese acceso compartido puede producir resultados inconsistentes.

Ejemplo inicial: productor y consumidor

Productor

```
// Initialize shared memory
shared_mem->count = 0;
while (fgets(input, BUFFER_SIZE, stdin) != NULL) {
    // Wait if buffer is full
    while (shared_mem->count > 0) {
        sleep(1);
    }
    // Copy input to shared memory
    strncpy(shared_mem->buffer, input, BUFFER_SIZE);
    shared_mem->count++;
}
```

Consumidor

```
// Wait for producer to write data
while (1) {
    // Check if there's data to consume
    if (shared_mem->count > 0) {
        printf("Consumed: %s", shared_mem->buffer);
        shared_mem->count = 0; // Mark as consumed
    }
    sleep(1); // Wait before checking again
}
```



Ejemplo inicial: productor y consumidor

Un proceso, **productor**, genera datos y los deposita en un búfer compartido. Otro proceso, **consumidor**, toma esos datos desde el mismo búfer para procesarlos.

Primero, **exclusión mutua**: ambos procesos no deberían modificar simultáneamente estructuras compartidas del búfer, porque podrían corromperse los datos.

Segundo, **sincronización por estado**: el productor no debe insertar elementos si el búfer está lleno, y el consumidor no debe extraer elementos si el búfer está vacío.

Es decir:

- Una cosa es **proteger** el búfer
- Y otra cosa es **coordinar** cuándo se puede producir o consumir

Versión del productor / consumidor

Un búfer de tamaño fijo y una variable compartida, `count`, que indica la cantidad de ítems.

El productor agrega un elemento y luego incrementa `count`.

El consumidor retira un elemento y luego decrementa `count`.

Visto por separado, cada uno de estos programas parece correcto:

- El productor no debería producir si el búfer está lleno
- El consumidor no debería consumir si el búfer está vacío

```
while (true) {  
    /* produce an item in next_produced */  
  
    while (count == BUFFER_SIZE)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    count++;  
}
```

PRODUTOR

```
while (true) {  
    while (count == 0)  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    count--;  
  
    /* consume the item in next_consumed */  
}
```

CONSUMIDOR

Versión del productor / consumidor

Como ambos procesos comparten variables y estructuras, aunque sean razonables en aislamiento, no garantiza que la combinación concurrente también lo sea.

En sistemas concurrentes, es un error asumir que la correctitud local implica correctitud global.

```
while (true) {  
    /* produce an item in next_produced */  
  
    while (count == BUFFER_SIZE)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    count++;  
}
```

PRODUTOR

```
while (true) {  
    while (count == 0)  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    count--;  
  
    /* consume the item in next_consumed */  
}
```

CONSUMIDOR

Dónde empieza el problema

En concurrencia, las instrucciones del productor y del consumidor se intercalan.

Si ambos acceden a la misma variable compartida, `count`, el valor final puede depender del instante exacto en que se produzca ese intercalado.

Aunque la lógica del productor sea correcta, y la del consumidor también, el sistema completo puede terminar en un estado inconsistente.

No es que alguno de los procesos “esté mal programado”. El problema es que el acceso compartido no está protegido.

```
while (true) {  
    /* produce an item in next_produced */  
  
    while (count == BUFFER_SIZE)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    count++;  
}
```

PRODUTOR

```
while (true) {  
    while (count == 0)  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    count--;  
  
    /* consume the item in next_consumed */  
}
```

CONSUMIDOR



Race condition

Al manipular datos compartidos, el resultado final depende del orden particular en que se ejecutan esas operaciones, eso es una **condición de carrera**.

Desde el punto de vista lógico, eso vuelve al sistema en **no determinista**. Es decir, un mismo programa puede producir resultados diferentes en distintas ejecuciones, aun cuando los datos de entrada sean los mismos.

Ese comportamiento es peligroso. Un error puede aparecer una vez y no volver a verse en la ejecución siguiente, aunque el código no haya cambiado.



Race condition

``count++`` y ``count--`` parecen simples, pero a nivel máquina implican varios pasos:

- leer
- modificar
- y escribir

Esto es:

- No son operaciones atómicas
- El intercalado puede romper la consistencia

Ejemplo

Supongamos inicialmente `count = 5`

- Productor ejecuta parte de `count++`
- Consumidor ejecuta parte de `count--`

Ambos trabajan sobre un valor viejo y un resultado final posible puede ser:

- 4 o
- 6

Cuando el resultado correcto esperado es:

- 5

```
register1 = count  
register1 = register1 + 1  
count = register1
```

```
register2 = count  
register2 = register2 - 1  
count = register2
```

T_0 :	<i>producer</i>	execute	<code>register₁ = count</code>	<code>{register₁ = 5}</code>
T_1 :	<i>producer</i>	execute	<code>register₁ = register₁ + 1</code>	<code>{register₁ = 6}</code>
T_2 :	<i>consumer</i>	execute	<code>register₂ = count</code>	<code>{register₂ = 5}</code>
T_3 :	<i>consumer</i>	execute	<code>register₂ = register₂ - 1</code>	<code>{register₂ = 4}</code>
T_4 :	<i>producer</i>	execute	<code>count = register₁</code>	<code>{count = 6}</code>
T_5 :	<i>consumer</i>	execute	<code>count = register₂</code>	<code>{count = 4}</code>

Ejemplo

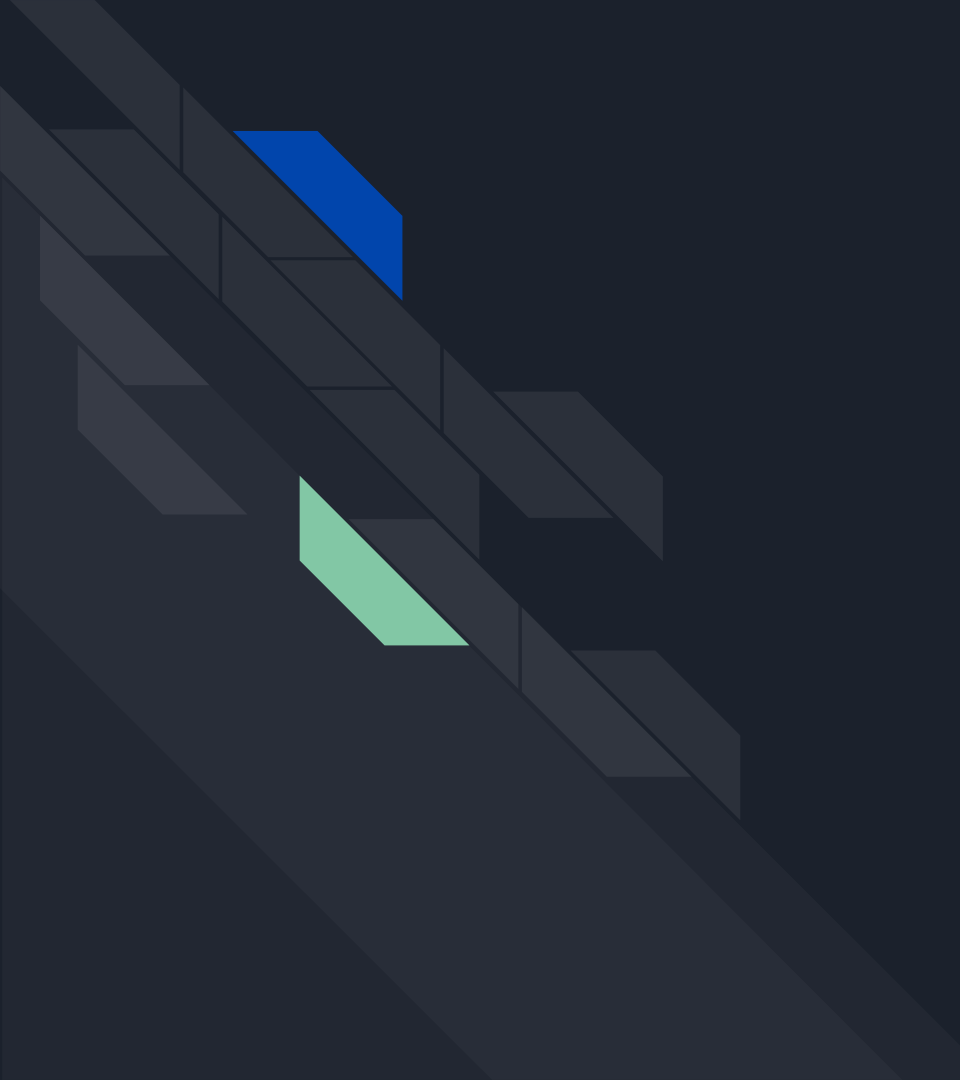
- El error no es matemático.
- El error es de **intercalado**.

Si el problema es el acceso concurrente a una parte sensible del código, entonces esa parte debe protegerse.

```
register1 = count  
register1 = register1 + 1  
count = register1
```

```
register2 = count  
register2 = register2 - 1  
count = register2
```

T_0 :	<i>producer</i>	execute	$register_1 = count$	$\{register_1 = 5\}$
T_1 :	<i>producer</i>	execute	$register_1 = register_1 + 1$	$\{register_1 = 6\}$
T_2 :	<i>consumer</i>	execute	$register_2 = count$	$\{register_2 = 5\}$
T_3 :	<i>consumer</i>	execute	$register_2 = register_2 - 1$	$\{register_2 = 4\}$
T_4 :	<i>producer</i>	execute	$count = register_1$	$\{count = 6\}$
T_5 :	<i>consumer</i>	execute	$count = register_2$	$\{count = 4\}$



REGIONES CRÍTICAS



Región crítica

Necesitamos identificar exactamente **qué parte del programa** es la que debe protegerse.

Región crítica o **Sección crítica**, es esa porción del código donde un proceso lee o modifica recursos compartidos.

La idea: si dos+ procesos ejecutan al mismo tiempo código que manipula el mismo dato compartido (**región crítica**), puede aparecer una inconsistencia. Entonces: **solo un proceso a la vez puede estar dentro de la región crítica**.

Todo el resto del programa, no necesita el mismo nivel de protección y puede ejecutarse en paralelo o intercalarse sin problema.

En el diseño concurrente, el desafío está en distinguir correctamente qué parte del programa debe ser considerada crítica y cuál no.



Qué debe garantizar una buena solución

Una solución al problema de la **exclusión mutua** debe cumplir varias condiciones.

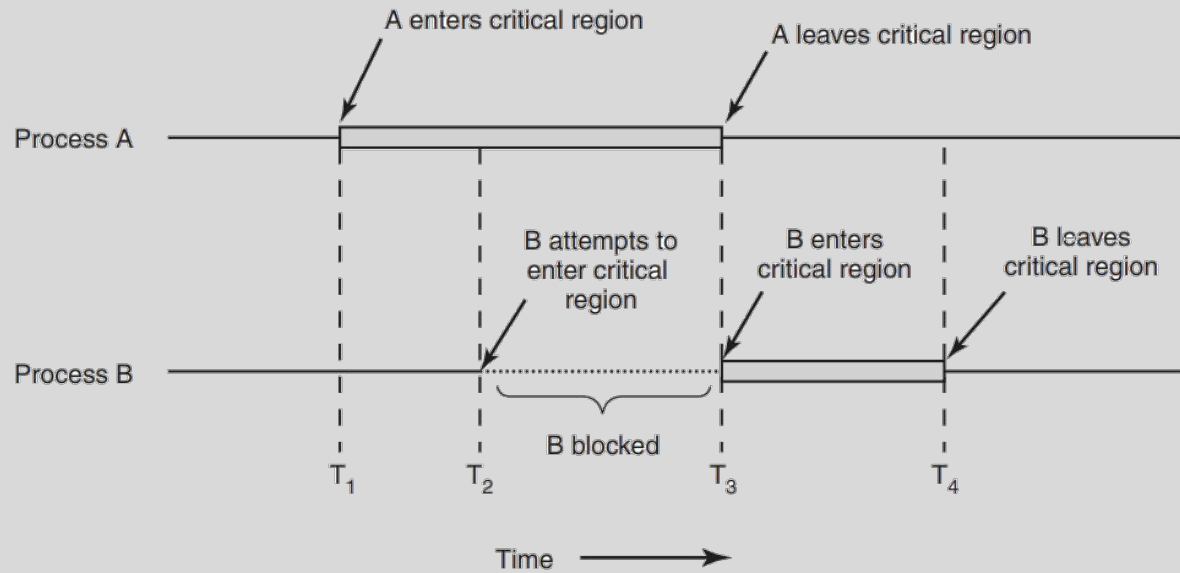
1. No pueden estar dos procesos al mismo tiempo dentro de sus regiones críticas. Esa es la condición de exclusión mutua propiamente dicha.
2. La solución debe garantizar **progreso**. Es decir, un proceso que está fuera de su región crítica no debería impedir innecesariamente que otro entre.
3. Debe evitarse la **espera indefinida**. Ningún proceso debería esperar para siempre si quiere entrar en su región crítica.
4. No se pueden hacer suposiciones sobre la velocidad ni el número de CPU. La solución tiene que seguir siendo válida aunque cambie el hardware o la planificación.

Comportamiento esperado

Un proceso entra en su región crítica y, mientras permanece ahí, cualquier otro proceso que quiera entrar **debe esperar**. No admite acceso concurrente.

El acceso al recurso compartido queda **serializado**: no porque los procesos no sean concurrentes, sino porque la **porción sensible** del trabajo sí debe ejecutarse de a uno.

La sincronización impone orden solamente en los puntos donde el desorden produciría errores.



Mutual exclusion using critical regions.

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```

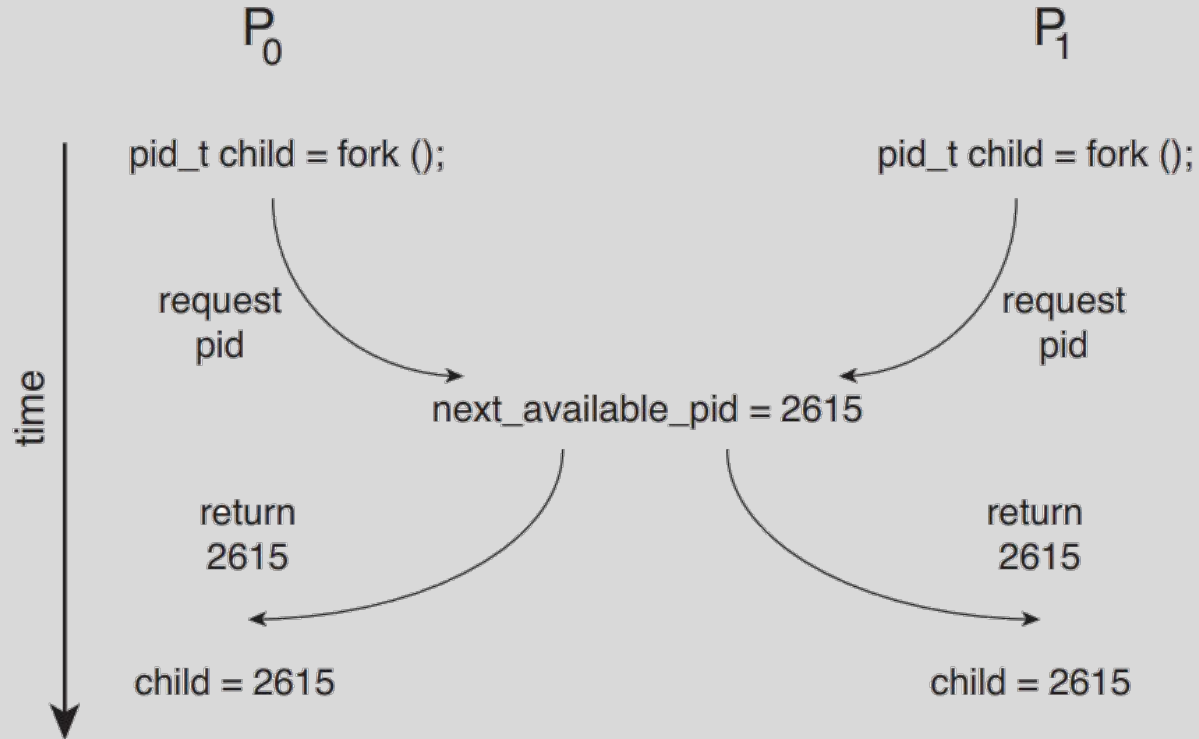
Ejemplo: asignación de PID

El problema de sincronización también aparece dentro del propio SO.

Si dos procesos ejecutan al mismo tiempo la *syscall* `fork()`. El kernel debe crear dos procesos hijos y asignarles un id único, por ejemplo `next_available_pid`

Si ambas ejecuciones leen el mismo valor de esa variable antes de actualizarla, podrían terminar asignando el mismo PID a dos procesos distintos.

Conclusión: el kernel también necesita sincronización.



Race condition when assigning a pid.

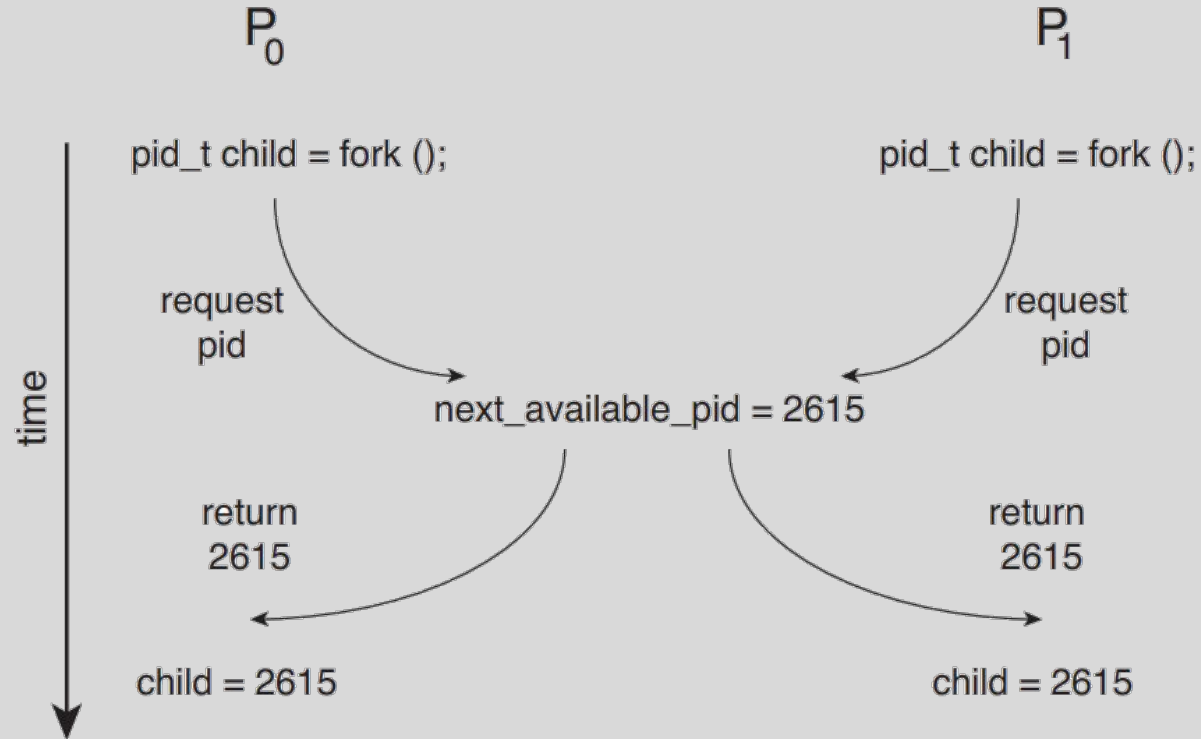
Ejemplo: asignación de PID

Otras estructuras críticas del kernel:

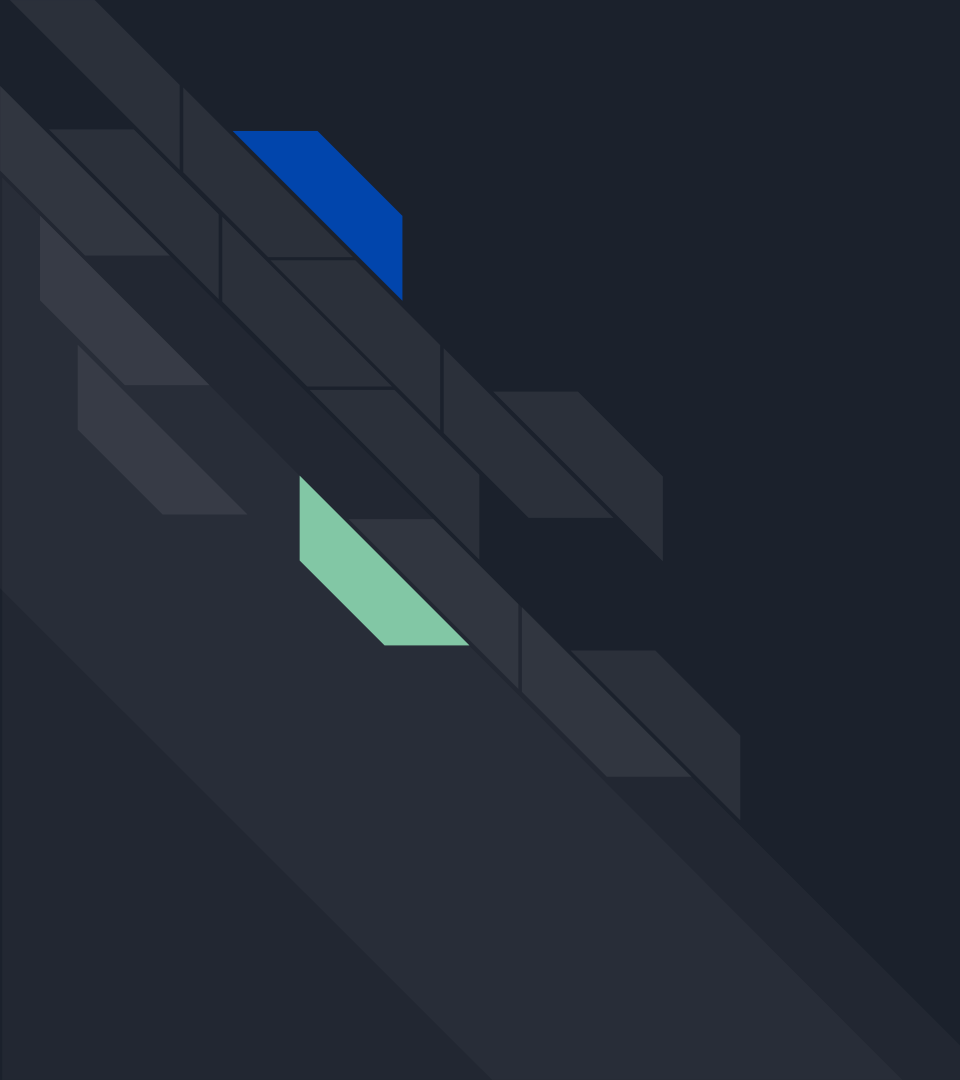
- Memoria
- Listas de procesos
- Archivos abiertos
- Interrupciones / Dispositivos
- Colas internas

Responsabilidad del kernel:

- Mantener consistencia
- Evitar *race conditions*



Race condition when assigning a pid.



SOLUCIONES A LA EXCLUSIÓN MUTUA



Primer intento: deshabilitar interrupciones

Una primera idea: deshabilitar las interrupciones mientras un proceso ejecuta su región crítica.

Como solución general tiene varios problemas:

1. No es aceptable permitir que procesos de usuario deshabiliten interrupciones libremente. Un proceso con errores podría dejarlas desactivadas indefinidamente y comprometer todo el sistema.
2. Esta técnica tampoco resuelve el problema en sistemas multinúcleo. Deshabilitar interrupciones en una CPU no impide que otra CPU siga ejecutando y accediendo a la misma estructura compartida.



Segundo intento: variable de bloqueo simple

Segunda idea: usar una variable compartida, `lock`, que indique si la región crítica está libre u ocupada.

La lógica es simple:

- Si `lock == 0`, el proceso lo cambia a 1 y entra a su región crítica.
- Si `lock == 1`, espera.

A primera vista parece suficiente, pero en realidad sigue teniendo el mismo problema de fondo.

Si dos procesos leen casi al mismo tiempo que `lock` vale 0, los dos podrían terminar entrando a la región crítica simultáneamente.

La variable de bloqueo no elimina la *race condition* si “verificar y tomar el lock” no es atómico.

Tercer intento: alternancia estricta y busy waiting

Usar una variable `turn` que indique quién tiene el turno para entrar en la región crítica. Por ejemplo:

- Si `turn == 0`, entra el proceso 0.
- Si `turn == 1`, entra el proceso 1.

El proceso que no tiene el turno, se queda en un bucle verificando continuamente si la variable cambió.

Espera ocupada (*busy waiting*): el proceso no avanza, pero tampoco se bloquea. Sigue consumiendo CPU mientras revisa una condición una y otra vez.

```
int turn = 0;

while (TRUE) {
    while (turn != 0)    /* loop */ ;
    critical_region();
    turn = 1;
    noncritical_region();
}
```

(a) Process 0.

```
while (TRUE) {
    while (turn != 1)    /* loop */ ;
    critical_region();
    turn = 0;
    noncritical_region();
}
```

(b) Process 1.

Tercer intento: alternancia estricta y busy waiting

Este enfoque efectivamente controla que entre uno u otro. Pero introduce dos problemas:

1. **Desperdicio de CPU:** un proceso puede pasar tiempo ejecutando un bucle sin hacer trabajo útil.
2. **Obliga a una alternancia rígida** (incluso cuando no hace falta): Un proceso puede quedar esperando su turno aunque el otro ni siquiera quiera entrar a la región crítica en ese momento.

Nota: Entender que puede bloquear el avance de un proceso aun cuando la región crítica está disponible.

```
int turn = 0;

while (TRUE) {
    while (turn != 0)    /* loop */ ;
    critical_region();
    turn = 1;
    noncritical_region();
}
```

(a) Process 0.

```
while (TRUE) {
    while (turn != 1)    /* loop */ ;
    critical_region();
    turn = 0;
    noncritical_region();
}
```

(b) Process 1.

Por qué la alternancia estricta no alcanza

Supongamos que el proceso 0 sale de su región crítica y deja `turn = 1`, habilitando al proceso 1.

Pero el proceso 1 no necesita entrar todavía en su región crítica, está haciendo otro trabajo en su sección **no crítica**. Mientras tanto, el proceso 0 termina rápido su trabajo no crítico y quiere volver a entrar y no puede hacerlo, porque `turn` sigue en 1.

Resultado: queda girando en **espera ocupada**, aun cuando el proceso 1 no está usando la **región crítica**.

Problema: rompe con la idea de progreso. El hecho de que un proceso esté fuera de su región crítica no debería impedir que otro entre.

```
int turn = 0;

while (TRUE) {
    while (turn != 0)    /* loop */ ;
    critical_region();
    turn = 1;
    noncritical_region();
}
```

(a) Process 0.

```
while (TRUE) {
    while (turn != 1)    /* loop */ ;
    critical_region();
    turn = 0;
    noncritical_region();
}
```

(b) Process 1.

Solución de Peterson (Software)

Es una solución de software al problema de la exclusión mutua entre dos procesos.

Utiliza dos elementos compartidos:

- Un arreglo `flag`, donde cada proceso indica si quiere entrar a su región crítica.
- Una variable `turn`, que se usa para resolver el conflicto cuando ambos quieren entrar al mismo tiempo.

```
#include <stdbool.h>

volatile bool flag[2] = {false, false};
volatile int turn;

void process(int i)
{
    int j = 1 - i;
    while (true)
    {
        flag[i] = true;
        turn = j;
        while (flag[j] && turn == j)
        {
            // Busy wait
        }
        // Critical section
        // ...
        flag[i] = false;
        // Remainder section
        // ...
    }
}

// Peterson's solution is a classic algorithm for mutual exclusion in concurrent
// programming. It allows two processes to share a single-use resource without conflict.
```

Solución de Peterson (Software)

La idea es, cuando un proceso quiere entrar:

1. Anuncia su intención poniendo su bandera en verdadero.
2. Cede el turno al otro proceso.
3. Espera solo si el otro también quiere entrar y además es su turno.

Con esta combinación, Peterson logra:

- No obliga a alternar siempre.
- Y resuelve correctamente el conflicto cuando ambos compiten al mismo tiempo.

```
#include <stdbool.h>

volatile bool flag[2] = {false, false};
volatile int turn;

void process(int i)
{
    int j = 1 - i;
    while (true)
    {
        flag[i] = true;
        turn = j;
        while (flag[j] && turn == j)
        {
            // Busy wait
        }
        // Critical section
        // ...
        flag[i] = false;
        // Remainder section
        // ...
    }
}

// Peterson's solution is a classic algorithm for mutual exclusion in concurrent
// programming. It allows two processes to share a single-use resource without conflict.
```

Solución de Peterson (Software)

Es una solución para **dos procesos**, no una receta general.

Históricamente es una solución muy importante porque muestra que, bajo ciertas condiciones, es posible resolver exclusión mutua por software sin depender directamente de instrucciones especiales de hardware.

Su valor es principalmente conceptual e histórico.

```
#include <stdbool.h>

volatile bool flag[2] = {false, false};
volatile int turn;

void process(int i)
{
    int j = 1 - i;
    while (true)
    {
        flag[i] = true;
        turn = j;
        while (flag[j] && turn == j)
        {
            // Busy wait
        }
        // Critical section
        // ...
        flag[i] = false;
        // Remainder section
        // ...
    }
}

// Peterson's solution is a classic algorithm for mutual exclusion in concurrent
// programming. It allows two processes to share a single-use resource without conflict.
```



Por qué la solución de Peterson sí funciona

1. Garantiza **exclusión mutua**: no permite que ambos procesos entren simultáneamente en sus regiones críticas.
2. Garantiza **progreso**: si un proceso no está interesado en entrar, no bloquea innecesariamente al otro. Y si ambos quieren entrar al mismo tiempo, el uso combinado de `flag` y `turn` resuelve el conflicto de manera ordenada.
3. Evita la **espera indefinida**: ninguno de los dos procesos puede quedar postergado para siempre si sigue intentando ingresar.

La solución de Peterson es muy valiosa desde el punto de vista teórico.

Pero también hay que ubicarla correctamente: no es la solución práctica habitual en software moderno, sino una herramienta conceptual para entender qué propiedades debe cumplir una solución correcta.

Soporte de hardware para exclusión mutua

Muchas arquitecturas incorporan **operaciones atómicas**, es decir, instrucciones que se ejecutan como una unidad indivisible. Mientras una CPU ejecuta una de estas operaciones sobre cierta ubicación de memoria, otra no puede interrumpir ese mismo acceso intermedio.

Resuelven el problema de:

- Verificar y tomar el lock por separado

Si el hardware permite realizar acciones de forma indivisible, entonces es posible construir mecanismos de exclusión mutua mucho más robustos.

enter_region:

```
TSL REGISTER,LOCK | copy lock to register and set lock to 1
CMP REGISTER,#0   | was lock zero?
JNE enter_region  | if it was not zero, lock was set, so loop
RET               | return to caller; critical region entered
```

leave_region:

```
MOVE LOCK,#0      | store a 0 in lock
RET               | return to caller
```

Entering and leaving a critical region using the TSL instruction.

Test and Set Lock (TSL)

Una de las instrucciones históricamente utilizadas para implementar exclusión mutua es **TSL**.

La idea es que la instrucción lee una variable de memoria asociada al lock y, en el mismo acto indivisible, escribe un valor que marca el lock como tomado.

Un proceso puede intentar adquirir el lock ejecutando TSL:

- Si el lock estaba libre, lo toma y entra en la región crítica.
- Si ya estaba ocupado, debe esperar y volver a intentar.

enter_region:

```
TSL REGISTER,LOCK | copy lock to register and set lock to 1
CMP REGISTER,#0   | was lock zero?
JNE enter_region  | if it was not zero, lock was set, so loop
RET               | return to caller; critical region entered
```

leave_region:

```
MOVE LOCK,#0      | store a 0 in lock
RET                | return to caller
```

Entering and leaving a critical region using the TSL instruction.



Test and Set Lock (TSL)

Vale mencionar que, a pesar de que TSL resuelve el problema de atomicidad en la adquisición del lock, si se utiliza en un bucle reintentando continuamente, todavía da lugar a **busy waiting**.

enter_region:

```
TSL REGISTER,LOCK
CMP REGISTER,#0
JNE enter_region
RET
```

```
| copy lock to register and set lock to 1
| was lock zero?
| if it was not zero, lock was set, so loop
| return to caller; critical region entered
```

leave_region:

```
MOVE LOCK,#0
RET
```

```
| store a 0 in lock
| return to caller
```

Entering and leaving a critical region using the TSL instruction.

Swap / XCHG

Otra instrucción que puede utilizarse con el mismo objetivo es **Swap** o **XCHG**.

Su función es intercambiar de manera atómica el contenido de un registro con el de una posición de memoria. Esa atomicidad permite implementar locks de forma similar a TSL.

El efecto es el mismo: el proceso intenta dejar marcado el lock como ocupado y, al mismo tiempo, obtener el valor previo para saber si estaba libre o no.

El punto importante no es la instrucción particular, sino la idea general: el hardware puede proporcionar primitivas atómicas sobre memoria compartida, y esas primitivas sirven como base para construir mecanismos de exclusión mutua correctos.

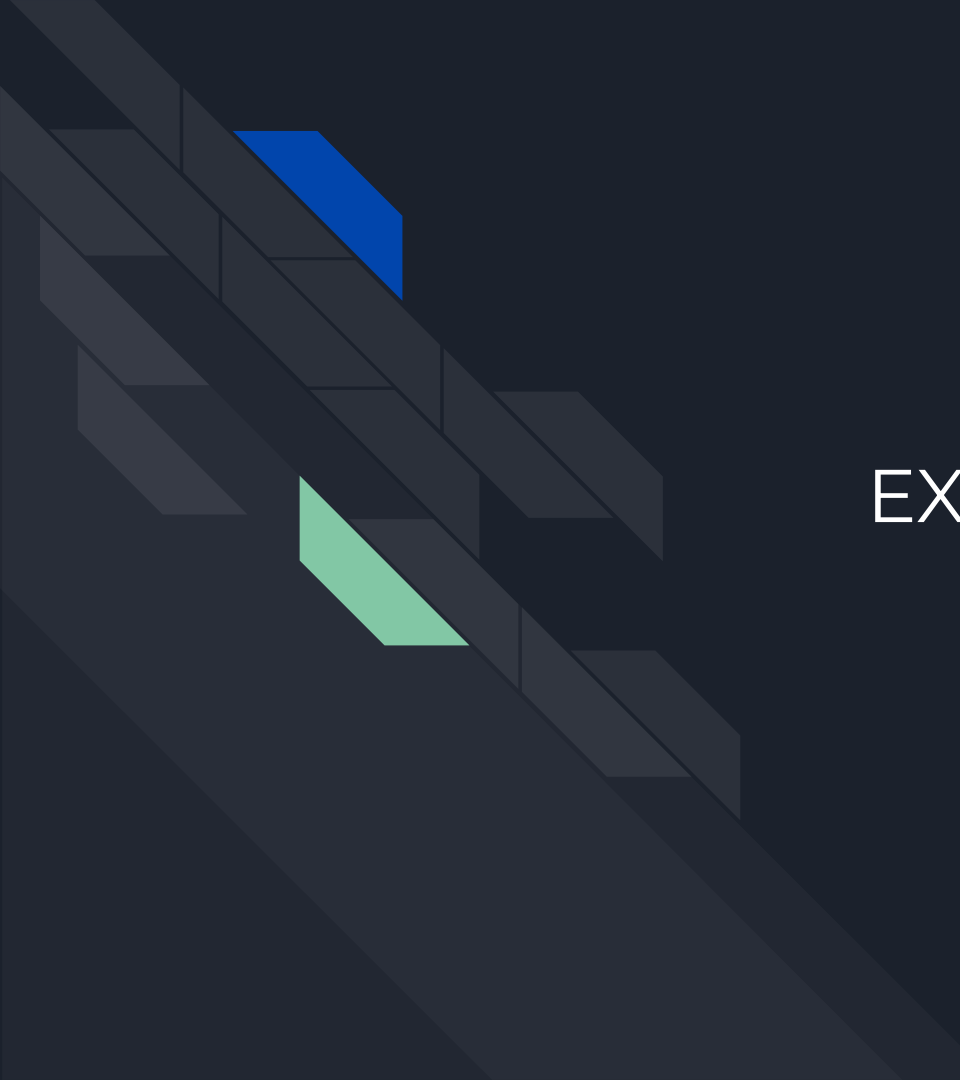
enter_region:

MOVE REGISTER,#1	put a 1 in the register
XCHG REGISTER,LOCK	swap the contents of the register and lock variable
CMP REGISTER,#0	was lock zero?
JNE enter_region	if it was non zero, lock was set, so loop
RET	return to caller; critical region entered

leave_region:

MOVE LOCK,#0	store a 0 in lock
RET	return to caller

Entering and leaving a critical region using the XCHG instruction.



EXCLUSION MUTUA SIN BUSY WAITING



El problema del busy waiting

Tanto la alternancia estricta como Peterson y los locks basados en TSL o XCHG comparten, en su forma más simple, una característica: cuando un proceso no puede entrar, se queda **esperando activamente**.

Eso significa que sigue ejecutándose, revisando una condición una y otra vez, aunque no esté realizando trabajo útil. Eso es **busy waiting**.

Tenemos una solución correcta desde el punto de vista **lógico**, pero no así desde el punto de vista de la eficiencia.

Una solución puede ser correcta y aun así estar lejos de lo óptimo.



Esperar bloqueado: sleep y wakeup

Idea: permitir que el proceso se **bloquee** cuando no puede continuar. Esto es:

- Si un proceso no puede entrar en su región crítica o no puede avanzar porque falta alguna condición, realiza una operación de **sleep** y queda suspendido.
- Cuando otro proceso modifica el estado de forma favorable, ejecuta una operación de **wakeup** para reactivarlo.

Pero, todavía falta ir un poco más allá. Si la coordinación entre *sleep* y *wakeup* no se maneja bien, también pueden aparecer errores de sincronización.



Semáforos: idea general

Para resolver estos problemas, aparece el concepto de **semáforo**.

Una variable entera especial que se manipula únicamente mediante operaciones atómicas bien definidas. Su función general es **coordinar el acceso entre procesos o hilos**.

Básicamente, el problema del lock simple es que solo puede expresar “libre” u “ocupado”. Un semáforo también puede **contar** recursos disponibles o eventos pendientes. Por eso sirve tanto para exclusión mutua como para **sincronización más general**.

Los procesos no modifican el semáforo de cualquier manera, solo a través de operaciones controladas por el sistema.



Operaciones `down` y `up`

La operación `down` intenta consumir una unidad del semáforo:

- Si el valor es mayor que cero, lo decrementa y continúa.
- Si el valor es cero, el proceso no puede seguir y queda bloqueado.

La operación `up` hace lo inverso:

- Incrementa el valor del semáforo.
- Si había procesos bloqueados esperando sobre ese semáforo, permite que alguno de ellos continúe.

El punto crucial es que estas operaciones deben ser **atómicas**.

El semáforo encapsula la sincronización dentro de operaciones indivisibles.

Productor-consumidor con semáforos

Usualmente se usan tres semáforos:

- `mutex`: para garantizar exclusión mutua sobre el búfer.
- `empty`: para contar cuántos espacios libres quedan.
- `full`: para contar cuántos ítems disponibles hay.

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

/ number of slots in the buffer */*
/ semaphores are a special kind of int */*
/ controls access to critical region */*
/ counts empty buffer slots */*
/ counts full buffer slots */*

/ TRUE is the constant 1 */*
/ generate something to put in buffer */*
/ decrement empty count */*
/ enter critical region */*
/ put new item in buffer */*
/ leave critical region */*
/ increment count of full slots */*

/ infinite loop */*
/ decrement full count */*
/ enter critical region */*
/ take item from buffer */*
/ leave critical region */*
/ increment count of empty slots */*
/ do something with the item */*

The producer-consumer problem using semaphores.

Productor-consumidor con semáforos

El productor:

- Espera lugar: `down (empty)`
- Entra al búfer: `down (mutex)`
- Produce
- Sale del búfer: `up (mutex)`
- Avisa que hay un ítem:
`up (full)`

El consumidor:

- Espera un ítem: `down (full)`
- Entra al búfer: `down (mutex)`
- Consume
- Sale del búfer: `up (mutex)`
- Avisa que hay lugar: `up (empty)`

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

/ number of slots in the buffer */*
/ semaphores are a special kind of int */*
/ controls access to critical region */*
/ counts empty buffer slots */*
/ counts full buffer slots */*

/ TRUE is the constant 1 */*
/ generate something to put in buffer */*
/ decrement empty count */*
/ enter critical region */*
/ put new item in buffer */*
/ leave critical region */*
/ increment count of full slots */*

/ infinite loop */*
/ decrement full count */*
/ enter critical region */*
/ take item from buffer */*
/ leave critical region */*
/ increment count of empty slots */*
/ do something with the item */*

The producer-consumer problem using semaphores.



Cierre práctico con mutex

- El semáforo `mutex` se usa para **exclusión mutua**: su función es impedir que dos procesos accedan simultáneamente al búfer.
- Los semáforos `full` y `empty` se usan para **sincronización**: expresan cuándo el productor o el consumidor deben detenerse y cuándo pueden continuar.

En concurrencia no todos los mecanismos cumplen el mismo papel. Algunos protegen recursos, otros coordinan eventos o disponibilidad.

Cuando solo necesitamos exclusión mutua, **mutex**. Un mutex representa básicamente un lock pensado para que un solo hilo o proceso entre a una región crítica a la vez.

En **Pthreads**, aparecen funciones concretas para trabajar con **mutex** y también con **variables de condición**, que permiten bloquear hilos hasta que cierta condición se cumpla.

Pthreads

Para sincronizar *threads*.

- `pthread_mutex_init`: crea un mutex.
- `pthread_mutex_destroy`: destruye un mutex.
- `pthread_mutex_lock`: intenta obtener el bloqueo y si no puede, se bloquea.
- `pthread_mutex_trylock`: intentar obtener el bloqueo y si no puede, falla. Esta llamada permite que un hilo realice una **espera activa**.
- `pthread_mutex_unlock`: desbloquea un mutex y libera exactamente un hilo si hay uno o más esperando.

Thread call	Description
<code>Pthread_mutex_init</code>	Create a mutex
<code>Pthread_mutex_destroy</code>	Destroy an existing mutex
<code>Pthread_mutex_lock</code>	Acquire a lock or block
<code>Pthread_mutex_trylock</code>	Acquire a lock or fail
<code>Pthread_mutex_unlock</code>	Release a lock

Some of the Pthreads calls relating to mutexes.

Pthreads

Las **variables de condición** permiten que los hilos se bloqueen si no se cumple alguna condición:

- `pthread_cond_init`: crea una variable de condición.
- `pthread_cond_destroy`: destruye una variable.
- `pthread_cond_wait`: bloquea al hilo que realiza la llamada hasta que otro hilo le envíe una señal.
- `pthread_cond_signal`: envía una señal para desbloquear un hilo bloqueado.
- `pthread_cond_broadcast`: se utiliza cuando hay varios hilos potencialmente bloqueados y esperando la misma señal.

Thread call	Description
<code>pthread_cond_init</code>	Create a condition variable
<code>pthread_cond_destroy</code>	Destroy a condition variable
<code>pthread_cond_wait</code>	Block waiting for a signal
<code>pthread_cond_signal</code>	Signal another thread and wake it up
<code>pthread_cond_broadcast</code>	Signal multiple threads and wake all of them

Some of the Pthreads calls relating to condition variables.

PRODUTOR - CONSUMIDOR

```
#include <stdio.h>
#include <pthread.h>

#define MAX 10 /* how many numbers to produce */

pthread_mutex_t the_mutex; /* used for mutual exclusion */
pthread_cond_t condc, condp; /* used for signaling */

int buffer = 0; /* buffer used between producer and consumer */

/* main function */
int main(int argc, char **argv)
{
    pthread_t pro, con;
    pthread_mutex_init(&the_mutex, 0);
    pthread_cond_init(&condc, 0);
    pthread_cond_init(&condp, 0);

    pthread_create(&con, 0, consumer, 0);
    pthread_create(&pro, 0, producer, 0);
    pthread_join(pro, 0);
    pthread_join(con, 0);

    pthread_cond_destroy(&condc);
    pthread_cond_destroy(&condp);
    pthread_mutex_destroy(&the_mutex);
}
```

```
/* producer and consumer are the two threads */
void *producer(void *ptr) /* produce data */
{
    int i;
    for (i = 1; i <= MAX; i++)
    {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer != 0)
            pthread_cond_wait(&condp, &the_mutex);
        buffer = i; /* put item in buffer */
        pthread_cond_signal(&condc); /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

/* consumer and producer are the two threads */
void *consumer(void *ptr) /* consume data */
{
    int i;
    for (i = 1; i <= MAX; i++)
    {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer == 0)
            pthread_cond_wait(&condc, &the_mutex);
        buffer = 0; /* take item out of buffer */
        pthread_cond_signal(&condp); /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}
```

Muchas Gracias

Jeremías Fassi

Javier E. Kinter

